Method Bodies

In a "regular" method definition, the method body starts on the line after the method declaration and continues until a matching `end`. Ruby does not require the method body to be indented, but standard code style does indent method bodies two characters:

```
def a_method_name(arg)
  puts arg
end
```

Starting in Ruby 3.0, you can create one-line methods with a different syntax, sometimes called an "endless method," because you don't need the `end` statement.

It's the method name, any arguments, an optional space, an `=` and the method body:

```
def a_method_name(arg) = puts arg
```

The right side of the equal sign is a single expression. If the method takes arguments, the argument list must be surrounded by parentheses (the parentheses are optional in a regular definition). There must be either parenthesis or a space between the method name and the equals sign, or else the parser will consider the equals sign to be part of the method name.

Sometimes you will want to define a method with no body, often because it's a method that will be fully defined by subclasses. While you can do that in two lines, you will sometimes see this idiom:

```
def a_method_name; end
```

The semicolon, which is rare in Ruby code, is used here as a separator between multiple expressions on the same line. If the method is called, it returns `nil`.

Method Names

Method names should begin with a lowercase letter or underscore followed by a combination of letters, digits, and underscores. You won't get an error if you start a method name with an uppercase letter, but when Ruby sees you calling the method, it might guess that it is a constant, not a method invocation, and as a result it may parse the call incorrectly. By convention, method names starting with an uppercase letter are in the `Kernel` module and are used for type conversion. The `Integer` method, for example, converts its parameter to an integer.

In addition to letters, digits, and underscores, a method name may end with one of `?`, `!`, or `=`.

Methods that return a boolean result (so-called predicate methods) are often named with a trailing `?`:

```
1.even?                 # => false
2.even?                 # => true
1.instance_of?(Integer) # => true
```

Methods that are "dangerous," or that modify their receiver, may be named with a trailing exclamation point, `!`. These are sometimes called *bang methods*, and are often paired with a "safe" version that does not end in an exclamation point. For instance, class `String` provides both `chop` and `chop!` methods. The first returns a modified string; the second modifies the receiver in place.